Lab Worksheet 3: Functions

CORE 103 Computational Thinking

Fall 2026

Instructions

- This LW must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *LW3_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**

Exercises

1. Late-Night Coffee Pickup Names

This question should be implemented in the file ***q1.py****.*

Hassan works part-time at Desibucks, the campus coffee stall. Students write their names quickly while rushing between classes, so the capitalization is often messy. To help staff quickly identify late-night pickup orders on the order screen, the system displays names for orders placed at or before 19:00 in lowercase, while names for orders placed after 19:00 are displayed in uppercase.

Task

Write a function, `format_name`, that takes `name` and `order_hour` as parameters.

The function must print `name` in uppercase when `order_hour > 19` and in lowercase when `order_hour <= 19`.

After defining the function, read the two input values and call the function.

Using Built-in Methods

For this question, you may use the following Python methods to change the capitalization of a name.

```
>>> sample_name = "WaQaR"
>>> sample_name.upper()
'WAQAR'
>>> sample_name.lower()
'waqar'
```

Constraints

- `name` is a non-empty string.

- `order_hour` is an integer representing the hour in 24-hour format.
- `0 <= order_hour <= 23`

Interaction

The input contains two lines:

- The first line contains `name`.
- The second line contains `order_hour`.

The program must print a single line containing the name in uppercase or lowercase according to the order time.

Sample

Input	Output
waQar 20	WAQAR
FatIMA 19	fatima

In the first case, `order_hour = 20`, so the order is placed after 19:00 and the function prints the name in uppercase.

In the second case, `order_hour = 19`, so the order is placed at 19:00 and the function prints the name in lowercase.

Python Implementation

Implement the Python code in **q1.py**.

2. Hostel Fan Speed Cycle

*This question should be implemented in the file **q2.py**.*

During a late-night group study session in the hostel common room, students use a small desk fan to stay comfortable. The fan has one button and four settings. Every button press moves the fan to the next setting. After `HIGH`, the next press brings the fan back to `OFF`.

Setting 0	Setting 1	Setting 2	Setting 3
OFF	LOW	MEDIUM	HIGH

The fan is currently at setting `current_setting`. During a software stress test, the controller sends `button_presses` button-press commands. Because a stress test may contain a very large number of commands, determine the final fan setting without processing every command one by one.

Task

Write a function, `fan_speed`, that takes `current_setting` and `button_presses` as parameters and returns the final fan setting as one of `'OFF'`, `'LOW'`, `'MEDIUM'`, or `'HIGH'`.

After defining the function, read the two input values, call the function, and print the returned value.

Constraints

- `current_setting` is an integer.
- `button_presses` is an integer.

- `0 <= current_setting <=3`
- `0 <= button_presses <=10**9`
- You are not allowed to use loops.

Interaction

The input contains two lines:

- The first line contains `current_setting`.
- The second line contains `button_presses`.

The program must print one of `OFF`, `LOW`, `MEDIUM`, or `HIGH`, representing the final fan setting.

Sample

Input	Output
3 1	OFF
0 11	HIGH

In the first case, the fan starts at setting 3 (`HIGH`). One button press moves it to setting 0 (`OFF`).

In the second case, the fan starts at setting 0 (`OFF`). After 11 button presses, the final setting is 3 (`HIGH`).

Python Implementation

Implement the Python code in **q2.py**.

3. Go Green Vehicle Test

*This question should be implemented in the file **q3.py**.*

The government is running a "Go Green" program to check the fuel efficiency of private vehicles.

During a test, a vehicle starts with a full fuel tank and travels `distance_km` kilometers. The tank is then refilled to full for `refill_cost` rupees, while fuel costs `fuel_price` rupees per liter.

The fuel used during the test is calculated as:

```
fuel_used_liters =refill_cost / fuel_price
```

The vehicle's fuel consumption rate, measured in liters per 100 kilometers (L/100 km), is then calculated as:

```
fuel_consumption_rate =100 * fuel_used_liters /distance_km
```

A lower fuel consumption rate means better fuel efficiency. The calculated rate is compared with `target_rate`:

- If the rate is greater than `target_rate`, the vehicle **missed** the target.
- If the rate is equal to `target_rate`, the vehicle **just met** the target.
- If the rate is less than `target_rate`, the vehicle **performed better than** the target.

Task

Write a function, `are_you_green`, that takes four parameters:

- `distance_km`
- `refill_cost`
- `fuel_price`
- `target_rate`

The function must calculate the vehicle's fuel consumption rate and print exactly one of the following forms:

- `Missed the target by X L/100 km.`
- `Just met the target.`
- `Performed better than the target by X L/100 km.`

When a difference `X` is printed, round it to at most two decimal places.

After defining the function, read the four input values and call the function.

Using a Built-in Function

For this question, you may use Python's built-in `round()` function to round a number.

```
>>> difference = 4.9567
>>> round(difference, 2)
4.96
>>> difference = 2.0
>>> round(difference, 2)
2.0
```

The second argument tells Python the maximum number of decimal places to round to.

Constraints

- `distance_km`, `refill_cost`, `fuel_price`, and `target_rate` are floating-point numbers.
- `distance_km > 10.0`
- `refill_cost > 0`
- `fuel_price > 0`
- `target_rate > 0`

Interaction

The input contains four lines:

- The first line contains `distance_km`, the distance traveled in kilometers.
- The second line contains `refill_cost`, the amount paid to refill the tank to full.
- The third line contains `fuel_price`, the fuel price in rupees per liter.
- The fourth line contains `target_rate`, the target fuel consumption rate in L/100 km.

The function prints the appropriate message described in the Task section.

Sample

Input	Output
50.0 500.0 100.00 8.00	Missed the target by 2.0 L/100 km.
50.0 500.0 100.00 10.00	Just met the target.
50.0 500.0 100.00 14.00	Performed better than the target by 4.0 L/100 km.

In the first case, the fuel consumed is $500.0/100.00 = 5.0$ liters and the fuel consumption rate is $100 \times 5.0/50.0 = 10.0$ L/100 km. Since $10.0 > 8.0$, the vehicle misses the target by 2.0 L/100 km.

In the second case, the fuel consumption rate is 10.0 L/100 km, which is equal to the target. Therefore, the vehicle just meets the target.

In the third case, the fuel consumption rate is 10.0 L/100 km. Since $10.0 < 14.0$, the vehicle performs better than the target by 4.0 L/100 km.

Python Implementation

Implement the Python code in **q3.py**.

4. Campus Phone Deal Rating

*This question should be implemented in the file **q4.py**.*

A group of first-year friends is looking for a second-hand phone before the semester gets busy. They want a phone that can handle online classes, group chats, maps, and late-night study playlists without becoming frustrating. Instead of trusting only the shopkeeper's opinion, they create a simple rating system.

For each phone, the friends run the same benchmark app. The app reports a speed score `speed_score` and an average app lag time `lag_time`. The phone specifications provide the battery capacity `battery_mah`. The phone is checked against the following criteria:

1. *Speed score* must be greater than 50.
2. *Average app lag time* must be less than 0.7 seconds.
3. *Battery capacity* must be greater than 4500 mAh.

The friends use a custom rating scale from 5 to 10. Based on these criteria, the phone is assigned a *phone rating*:

- Rating 10 if all three conditions are met.
- Rating 9 if conditions 1 and 2 are met, but condition 3 is not.
- Rating 8 if conditions 2 and 3 are met, but condition 1 is not.
- Rating 7 if conditions 1 and 3 are met, but condition 2 is not.
- Rating 6 if exactly one condition is met.

- Rating 5 if none of the conditions are met.

For example, a phone with `speed_score = 60`, `lag_time = 0.9`, and `battery_mah = 4000` satisfies condition 1 only, so its rating is 6.

Task

Write a function, `phone_rating`, that takes `speed_score`, `lag_time`, and `battery_mah` as parameters and returns the corresponding phone rating.

After defining the function, read the three input values, call the function, and print the returned rating.

Constraints

- `speed_score` is an integer.
- `lag_time` is a floating-point number.
- `battery_mah` is an integer measured in mAh.
- `0 < speed_score <= 100`
- `0 <= lag_time <= 5`
- `1000 <= battery_mah <= 10000`

Interaction

The input contains three lines:

- The first line contains `speed_score`.
- The second line contains `lag_time`.
- The third line contains `battery_mah`.

The program must print a single integer representing the phone rating.

Sample

Input	Output
60 0.5 5000	10
50 0.7 4500	5

In the first case, all three conditions are satisfied, so the function returns 10.

In the second case, each value is exactly at its threshold. Since all three conditions use strict inequalities, none of the conditions are satisfied, so the function returns 5.

Python Implementation

Implement the Python code in **q4.py**.

5. Orientation Escape Room Decoder

*This question should be implemented in the file **q5.py**.*

During orientation week, first-year students take part in a campus escape-room challenge. At one checkpoint, they find a five-digit clue number `clue_number`.

To unlock the next checkpoint, the keypad checks the clue by calculating a verification score from the five digits of the clue number. Starting from the leftmost digit, the first digit is

multiplied by 1, the second by 2, the third by 3, the fourth by 4, and the fifth by 5. The verification score is the sum of these weighted values.

Task

Write a function, `decode_clue`, that takes the five-digit integer `clue_number` as a parameter and returns its verification score.

Use arithmetic operations to solve the problem. You may use integer division (`//`) and remainder (`%`).

After defining the function, read `clue_number`, call the function, and print the returned verification score.

Constraints

- `clue_number` is an integer.
- `10000 <= clue_number <= 99999`
- Do not use loops.
- Do not convert the number to a string.

Interaction

The input contains one line containing the five-digit integer `clue_number`.

The program must print a single integer representing the verification score.

Sample

Input	Output
58324	58
91736	74

In the first case, the five digits contribute `5*1 + 8*2 + 3*3 + 2*4 + 4*5 = 58`, so the function returns 58.

In the second case, the five digits contribute `9*1 + 1*2 + 7*3 + 3*4 + 6*5 = 74`, so the function returns 74.

Python Implementation

Implement the Python code in **q5.py**.

6. Temperature Confusion

This question should be implemented in the file ***q6.py****.*

Fatima studies in Pakistan and often talks to her best friend Sana, who is studying in the United States. Whenever they discuss the weather during video calls, both of them get confused because Fatima usually thinks in Celsius while Sana usually thinks in Fahrenheit.

To avoid doing the conversion manually every time, Fatima wants to create a small temperature converter. The converter receives a temperature value `temperature` and a letter `target_scale`. The value `target_scale` tells which temperature system the value should be converted *to*.

If `target_scale` is `'C'`, the given temperature is in Fahrenheit and must be converted to Celsius for Fatima. If `target_scale` is `'F'`, the given temperature is in Celsius and must be converted to Fahrenheit for Sana.

Temperature Conversion

The relationship between Celsius and Fahrenheit is

$$F = C \times \frac{9}{5} + 32,$$

where C represents a temperature in Celsius and F represents a temperature in Fahrenheit. You may rearrange this relationship as needed for the required conversion.

Task

Write a function, `temp_converter`, that takes `temperature` and `target_scale` as parameters and prints the converted temperature with the required message.

Your solution must also use the following helper functions:

- `fahrenheit_to_celsius(fahrenheit_temp)`, which returns the temperature converted to Celsius.
- `celsius_to_fahrenheit(celsius_temp)`, which returns the temperature converted to Fahrenheit.

The helper functions should return the converted numerical value. The `temp_converter` function should round the converted value to at most two decimal places before printing it.

If `target_scale` is `'C'`, print the converted value followed by the text "degrees Celsius is the temperature for Fatima."

If `target_scale` is `'F'`, print the converted value followed by the text "degrees Fahrenheit is the temperature for Sana."

After defining all three functions, read the two input values and call `temp_converter`.

Using a Built-in Function

For this question, you may use Python's built-in `round()` function to round a number.

For example:

```
>>> converted_temp = 23.4567
>>> round(converted_temp, 2)
23.46
>>> converted_temp = 35.0
>>> round(converted_temp, 2)
35.0
```

The second argument specifies the maximum number of decimal places to which the value is rounded.

Constraints

- `temperature` is an integer.
- `target_scale` is either `'C'` or `'F'`.
- If `target_scale == 'C'`, then `temperature` is in Fahrenheit and `-58 <= temperature <= 122`.
- If `target_scale == 'F'`, then `temperature` is in Celsius and `-50 <= temperature <= 50`.

Interaction

The input contains two lines:

- The first line contains the integer `temperature`.
- The second line contains `target_scale`, the scale to which the temperature must be converted.

The `temp_converter` function prints one line in the exact format described in the Task section.

Sample

Input	Output
95 C	35.0 degrees Celsius is the temperature for Fatima.
30 F	86.0 degrees Fahrenheit is the temperature for Sana.

In the first case, `target_scale = 'C'`, so 95 is treated as Fahrenheit and converted to 35.0 degrees Celsius for Fatima.

In the second case, `target_scale = 'F'`, so 30 is treated as Celsius and converted to 86.0 degrees Fahrenheit for Sana.

Python Implementation

Implement the Python code in **q6.py**.

Rough Work